



# MONGODB

Your first noSQL systems

1

Document  
based model

2

Design pattern

3

syntax

4

Efficiency

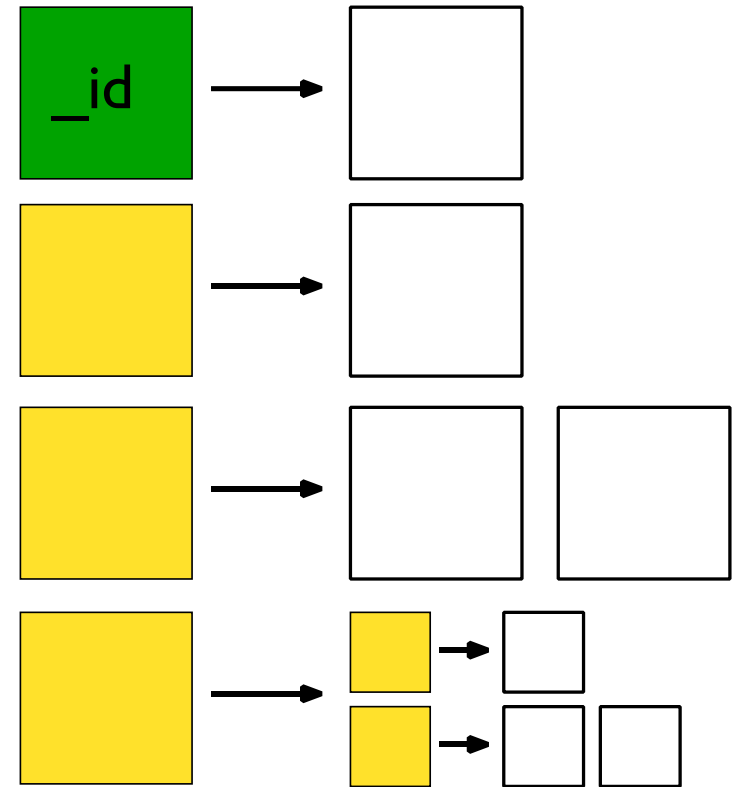
5

Query language

# DOCUMENT

- N-dimensional storage
- Each field can contain 0, 1,
- Query on **any field & level**
- **Flexible** schema

**many, or embedded** values



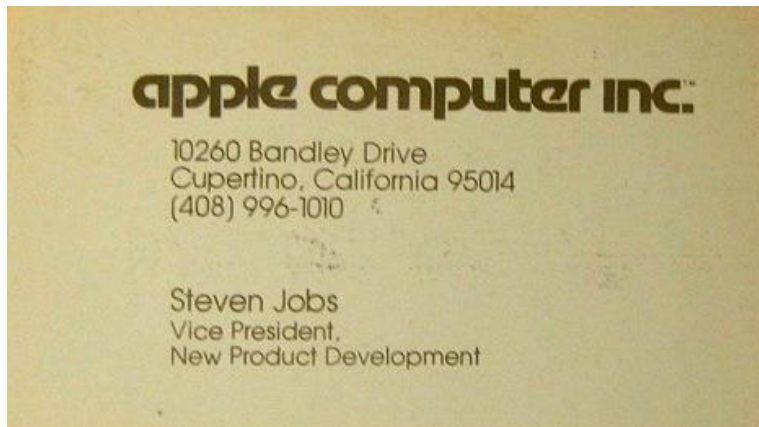
# MODELLING ISSUE

- The following examples will be based on document based model and a tool called mongodb as example, but they can be generalized to other models and tools

# REFERENCING

## Contacts

```
{  
  "_id": 2,  
  "name": "Steven Jobs",  
  "title": "VP, New Product  
Development",  
  "company": "Apple Computer",  
  "phone": "408-996-1010",  
  "address_id": 1  
}
```



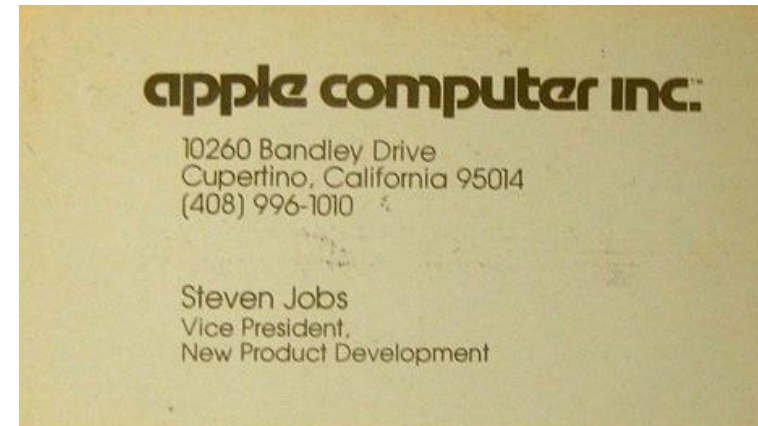
## Addresses

```
{  
  "_id": 1,  
  "street": "10260 Bandley  
Dr",  
  "city": "Cupertino",  
  "state": "CA",  
  "zip_code": "95014",  
  "country": "USA"  
}
```

# EMBEDDING

## Contacts

```
{  
  "_id": 2,  
  "name": "Steven Jobs",  
  "title": "VP, New Product  
Development",  
  "company": "Apple Computer",  
  "address": {  
    "street": "10260 Bandley Dr",  
    "city": "Cupertino",  
    "state": "CA",  
    "zip_code": "95014",  
    "country": "USA"  
  },  
  "phone": "408-996-1010"  
}
```





## Contact

- name
- company
- title
- phone

## Address

- street
- city
- state
- zip\_code

# Relational Schema

# Document Schema

## Contact

- name
- company
- address
  - street
  - city
  - State
  - zip\_code
- title
- phone



## Contact

- name
- company
- title
- phone

## Address

- street
- city
- state
- zip\_code

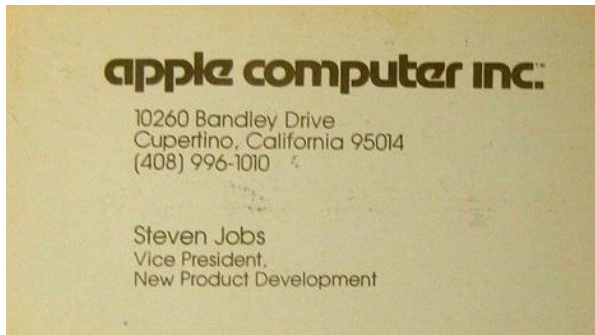
## Contact

- name
- company
- address
  - street
  - city
  - state
  - zip\_code
- title
- phone

**How are they different? Why?**

# SCHEMA FLEXIBILITY (ALSO FOR OTHER NOSQL MODELS)

```
{  
  "name": "Steven Jobs",  
  "title": "VP, New Product Development",  
  "company": "Apple Computer",  
  "address": {  
    "street": "10260 Bandley Dr",  
    "city": "Cupertino",  
    "state": "CA",  
    "zip_code": "95014"  
  },  
  "phone": "408-996-1010"  
}
```



```
{  
  "name": "Larry Page",  
  "url": "http://google.com/",  
  "title": "CEO",  
  "company": "Google!",  
  "email": "larry@google.com",  
  "address": {  
    "street": "555 Bryant, #106",  
    "city": "Palo Alto",  
    "state": "CA",  
    "zip_code": "94301"  
  },  
  "phone": "650-618-1499",  
  "fax": "650-330-0100"  
}
```



# MONGODB

- It is a document based management system
- Data are stored in Bson
- Access data thanks to indexes
- No join

| SQL vs NoSQL |            |
|--------------|------------|
| Table        | Collection |
| Row          | Document   |
| Column       | Key        |

# MONGODB

| RDBMS       |   | MongoDB           |
|-------------|---|-------------------|
| Database    | ➡ | Database          |
| Table, View | ➡ | Collection        |
| Row         | ➡ | Document (BSON)   |
| Column      | ➡ | Field             |
| Index       | ➡ | Index             |
| Join        | ➡ | Embedded Document |
| Foreign Key | ➡ | Reference         |
| Partition   | ➡ | Shard             |

# MONGODB

- Download
  - <https://www.mongodb.com/download-center#community>
- MongoDB Management tool
  - Studio 3t
    - <https://studio3t.com/download-now/>
  - MongoDB Compass
    - Available within mongo installer
- MongoDB connector BI



EFFICENCY



# MODEL VS EFFICENCY

- Large textual CSV file
  - for each row
    - StudentBadge, degree, [exam\*]
  - Students may have more than one exam
  - An exam may be obtained by several students
  - A student belongs to exactly one degree
- Question:
  - I need a list of exams with students and their number

Student(id,name,Surname,Degree)

S2E(Student,exam)

Exam(id, course, CFU, teacher)

# MODEL VS EFFICENCY

- How to model in JSON?

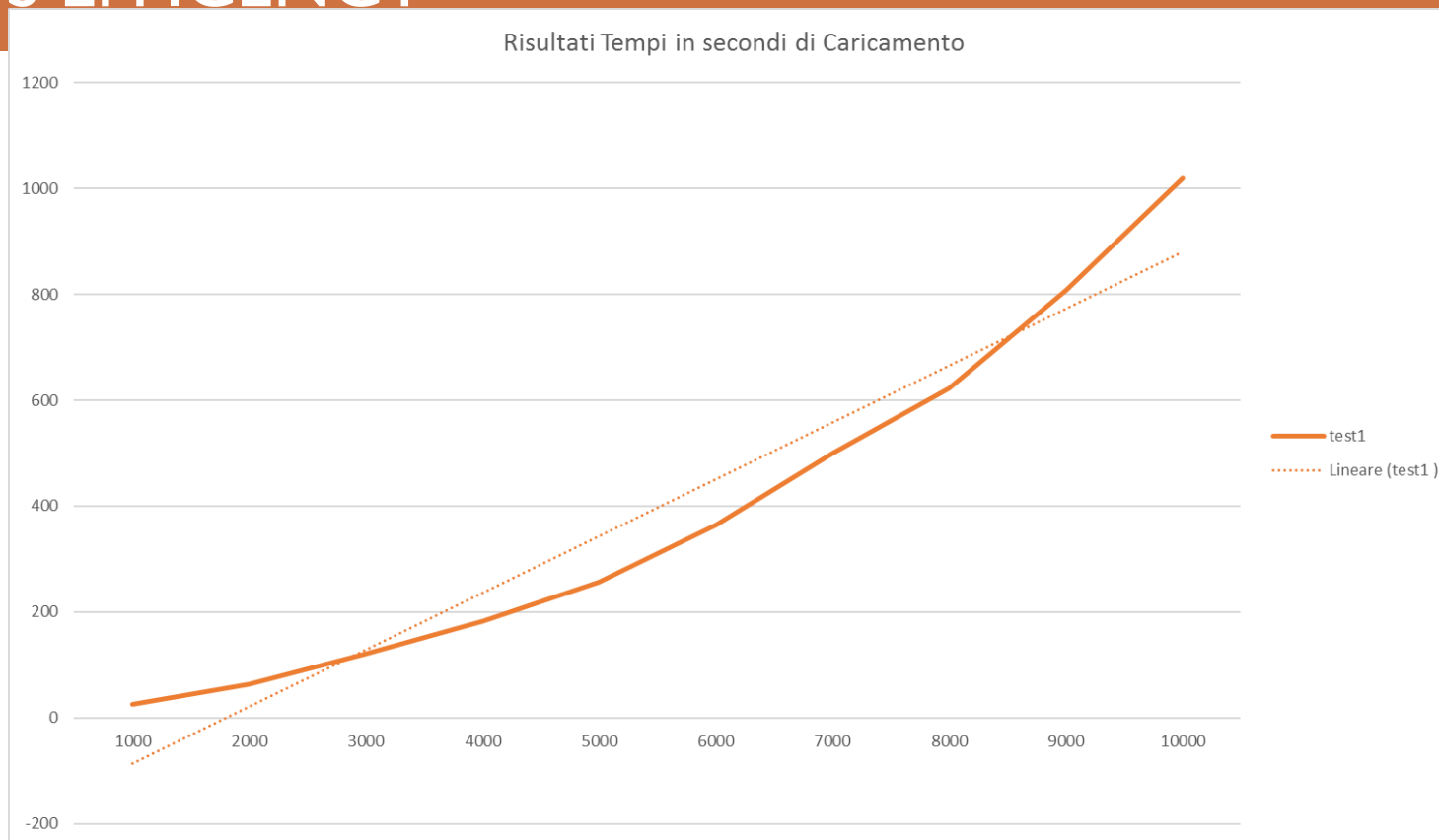
```
{  
  idExam: String  
  CourseName: String  
  CFU: Integer  
  Teacher: String  
  Students [{  
    Id:String  
    Name: String  
    Surname: String  
    MasterDegree: String  
  }*  
]  
  NumberOfStudents: Integer  
}
```



# MODEL VS EFFICENCY

- To load data we used a java program with 10 threads that open the same file and write on the same mongodb
- For each raw
  - Create a new document or
  - If exist the exam, the program increment the attribute NumberStudent
- in SQL there is no standard way to say
  - Insert if not exist
    - In MYSQL Insert ignore, duplicate..
- Hd infrastructure
  - My laptop CPU I7 3Ghz, 32GB Ram, 256GB SSD+1Thd
- How many seconds are needed to load 10.000 raw?

# MODEL VS EFFICENCY



Test I 17 minutes

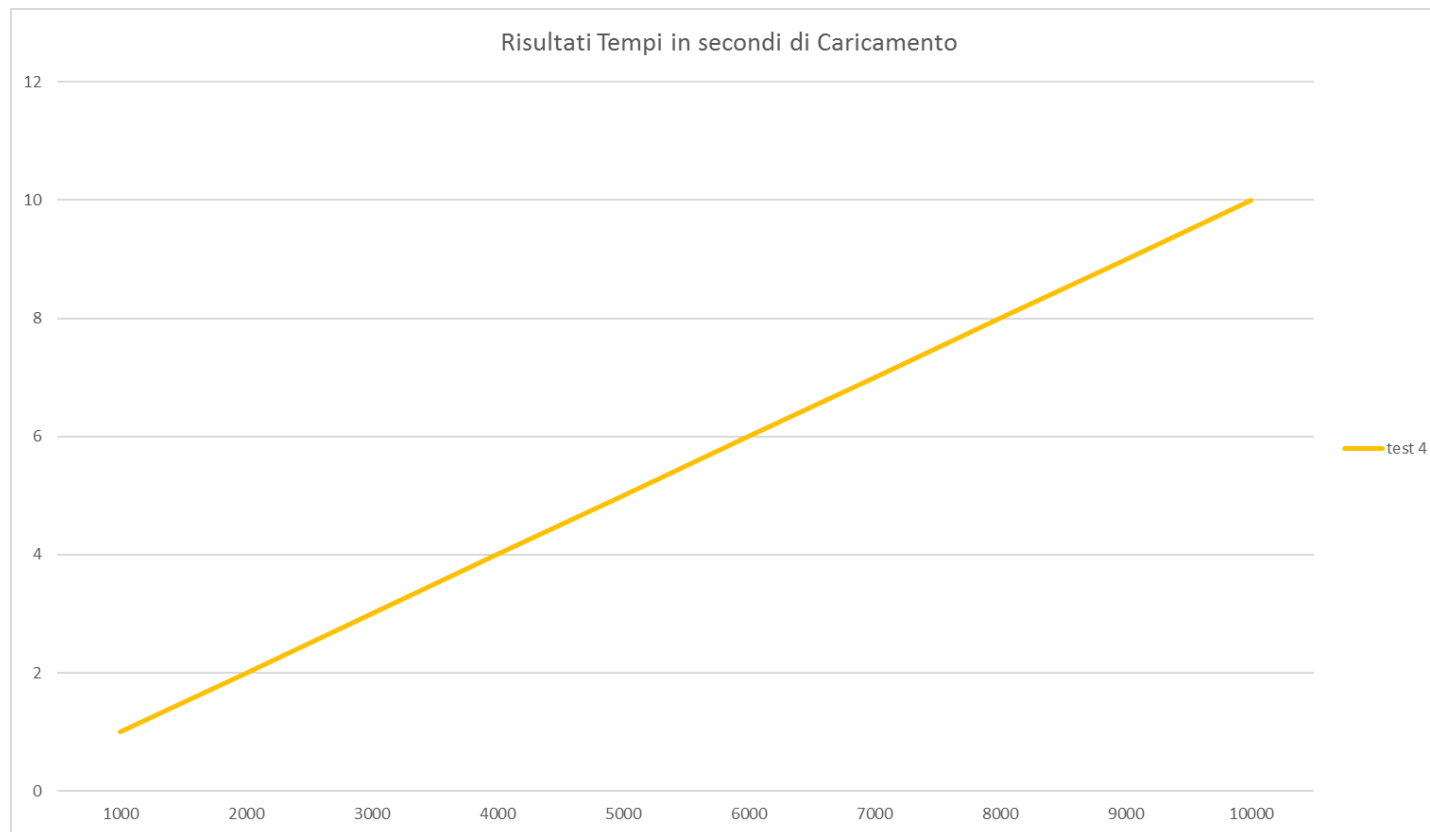
# MODEL VS EFFICENCY

- Change how to model data

```
{  
  IdEsame: String  
  Insegnamento: String  
  CFU: Integer  
  Docente: String  
  Matricola: String  
  Nome: String  
  Cognome: String  
  CDL: String  
}
```

```
Badge l, «datascience», ex1, ex2, ex3... →  
{badge l, «datascience», ex1}  
{badge l, «datascience», ex2}  
{badge l, «datascience», ex3}  
....
```

# MODEL VS EFFICENCY



Test4 10 secondi

# MODEL VS EFFICENCY

- How to calculate number of exams?

- SQL

Select studente, CDL, count(\*)

From S2E JOIN Studente on (studente=esame)

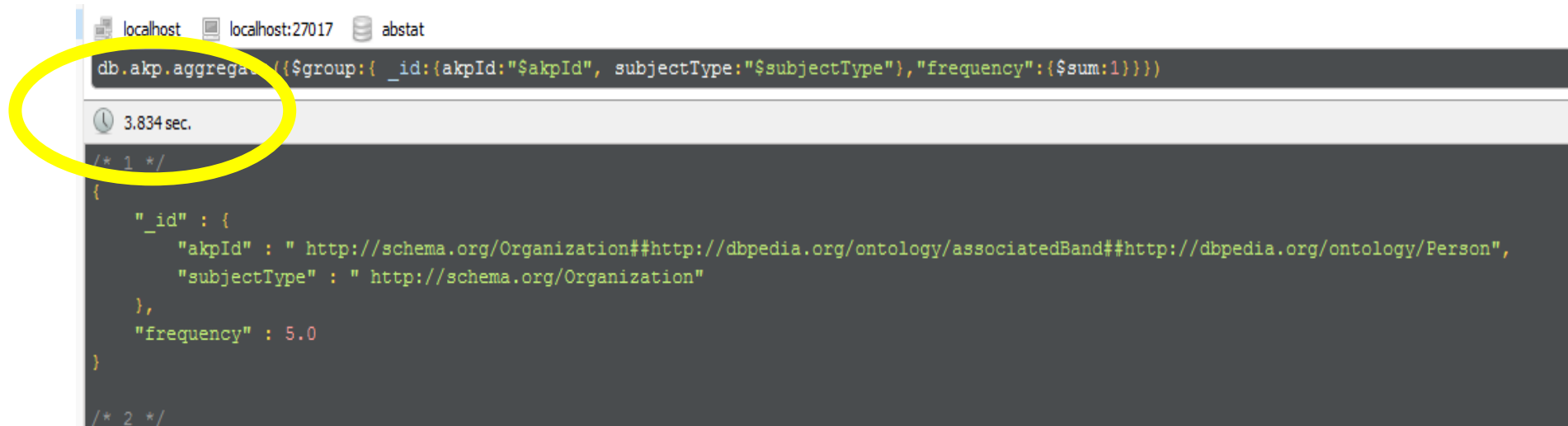
Grup by studente, CDL

- in MongoDB

```
db.akp.aggregate({$group:{_id:{idEsame:"$idEsame", student:"$student"},"risultato":{"$sum:1}}})
```

- 1.863.002 documents → query executed in

# MODEL VS EFFICENCY



```
localhost localhost:27017 abstat
db.akp.aggregate({$group:{_id:{akpId:"$akpId", subjectType:"$subjectType"},"frequency":{$sum:1}}})
3.834 sec.
/* 1 */
{
  "_id" : {
    "akpId" : " http://schema.org/Organization##http://dbpedia.org/ontology/associatedBand##http://dbpedia.org/ontology/Person",
    "subjectType" : " http://schema.org/Organization"
  },
  "frequency" : 5.0
}
/* 2 */
```



CREATE AND MANIPULATE DATA



# CREATE DATA

- Each document needs to be stored in a collection
  - A collection needs to be stored in a database
  - Create database (or select an existing one)
  - Create collection
- if you have an external source you can use mongoimport utility
- (SHELL) `Mongoimport -d database -c collection --type csv --ignoreblanks --file PATH`
- But....pay attention to details...



# INSERT

```
db.CollectionName.insertOne({  
  name:" Andrea" ,  
  age: 26,  
  Teach [«datawarehouse», «architecture»]  
})
```

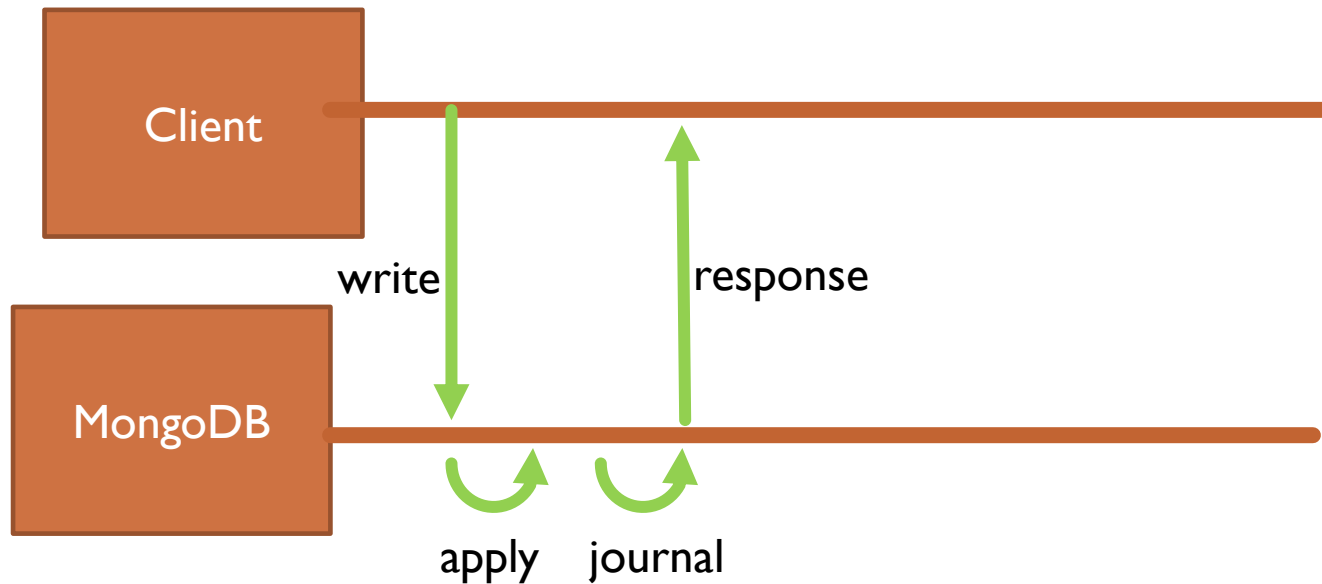
## Understanding the Document Model.

```
var p = {  
  '_id': '3432',  
  'author': DBRef('User', 2),  
  'title': 'Introduction to MongoDB',  
  'body': 'MongoDB is an open sources.. ',  
  'timestamp': Date('01-04-12'),  
  'tags': ['MongoDB', 'NoSQL'],  
  'comments': [{ 'author': DBRef('User', 4),  
                  'date': Date('02-04-12'),  
                  'text': 'Did you see.. ',  
                  'upvotes': 7, ... }  
  ]  
}  
> db.posts.save(p);
```



# MASSIVE DATA UPLOAD

- In case of large injection
  - Disable ack of data
  - Do not write on log file
    - Warning !!!



# UPDATE

- Update all documents within the CollectionName where name is equal to «Andrea», by setting tole attribute equals to «PA»

```
db.collectionName.updateAll(  
{  
  name:{$eq: «Andrea»},  
  {$set: {Role: «PA»}},  
  {$upsert:true}  
}  
)
```



If set to true, creates a new document when no document matches the query criteria. Optional

# DELETE

- Delete one/all document that matches a given query

```
db.CollectionName.deleteOne(  
    { status=«D»}  
)
```

```
db.CollectionName.deleteMany({})
```

Delete all documents not collection!

```
db.CollectionName.drop()
```



QUERY DATA



# DATA EXPLORATION

- To see all documents in the collection
- `db.nomeColl.find().pretty()`
  - `Pretty()` is useful to see human readable results
- To see the schema
  - Compass
  - Quality plugin (<https://github.com/maurino1973/QualityMongoData>)
  - ...

## SIMPLE QUERY

- `db.CollectionName.find(jsonQuery,jsonSelectList).Function()`
- Eg. Find all **users** that have **more than 18 years old** and **show me the name and the address** of the **first 5**

```
db.user.find({age:{$gt:18}},{name:1,address:1}).limits(5)
```

In sql

```
SELECT name, address
```

```
FROM user
```

```
WHERE age>18
```

```
LIMIT 5
```

# QUERY OPERATORS

| Operation           | Syntax                  | Example                                         | RDBMS Equivalent             |
|---------------------|-------------------------|-------------------------------------------------|------------------------------|
| Equality            | {<key>:<value>}         | db.mycol.find({"by":"tutorials point").pretty() | where by = 'tutorials point' |
| Less Than           | {<key>:{\$lt:<value>}}  | db.mycol.find({"likes":{\$lt:50}}).pretty()     | where likes < 50             |
| Less Than Equals    | {<key>:{\$lte:<value>}} | db.mycol.find({"likes":{\$lte:50}}).pretty()    | where likes <= 50            |
| Greater Than        | {<key>:{\$gt:<value>}}  | db.mycol.find({"likes":{\$gt:50}}).pretty()     | where likes > 50             |
| Greater Than Equals | {<key>:{\$gte:<value>}} | db.mycol.find({"likes":{\$gte:50}}).pretty()    | where likes >= 50            |
| Not Equals          | {<key>:{\$ne:<value>}}  | db.mycol.find({"likes":{\$ne:50}}).pretty()     | where likes != 50            |



# QUERY OPERATION

- To query documents based on the OR condition, you need to use \$or keyword. (AND is a “&”)
- `db.CollectionName.find(  
 { “likes”: { $gt: 10 },  
 $or: [ { “by”: “XXX” },  
 { “title”: “MongoDB books” }  
 ]  
}).pretty()`

# FOR LARGE DATASET

- In case of large dataset it is useful to use some additional structures called index
  - like the analytical indexes of the books, so the indexes provide a faster system to access the data
  - It can require some time during the insertion of data
- A primary index is always defined for the id

## Secondary Indexes

Create Index on any field in the document

// 1 means ascending, -1 means descending

```
> db.posts.ensureIndex({'author': 1});
```

//Index Nested Documents

```
> db.posts.ensureIndex('comments.author': 1);
```

// Index on tags

```
> db.posts.ensureIndex({'tags': 1});
```

// Geo-spatial Index

```
> db.posts.ensureIndex({'author.location': '2d'});
```



# FIND DOCUMENTS

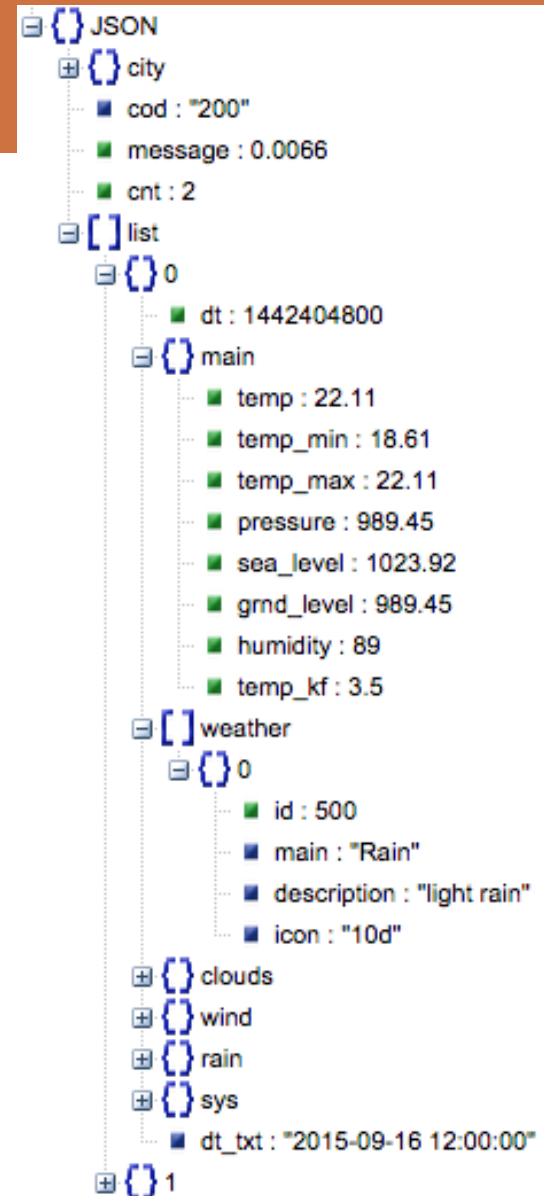
- `Db.nomeColl.find(opz)`
  - Opf: json document of the request
  - `{<field>:<value>,..}`
- Simple query
  - `Db.smallcoll.find({'«countrycode»: «TN»})`
    - Show all value of attribute whose countrycode is TN
- Query with not equal condition
  - `Db.smallcoll.find({'«countrycode»: {$in:[«TN», «IN»]}})`

# NESTED DOCUMENT

- `db.smallColl.find( {"theme1" : {  
    "Name" : "Education for all",  
    "Percent" : 100}})`
- `Db.smallColl.find(«theme1.Percent» : {$gt: 40})`

# SHOW RESULTS

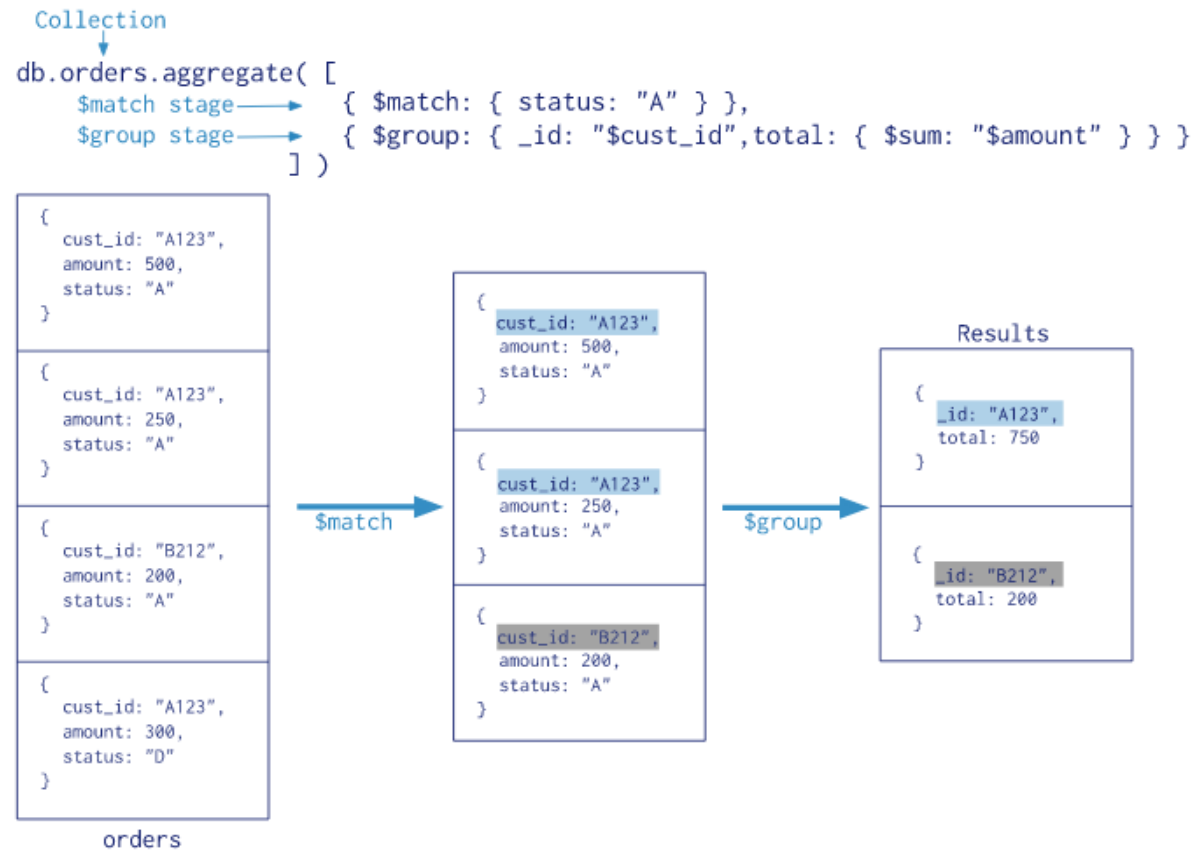
- `Db.smallColl.find({},{«countrycode»:1})`



# AGGREGATION

- Calculates aggregate values for the data in a collection or a view.
- `db.collection.aggregate(pipeline, options)`
  - Pipeline. A sequence of data aggregation operations or stages.
  - Options. Additional options that `aggregate()` passes to the aggregate command.

# AGGREGATION



# AGGREGATION

- The aggregation pipeline starts processing the documents of the collection and pass the result to the next Pipeline Operator in order to get the final result.
- The operators can filter out documents (e.g. \$match) generate new documents (e.g. \$group) computing the result from the given ones.
- The same operator can be used more than once in the pipeline.
- Each operator takes as input a pipeline expression that is a document itself containing:
  - Fields
  - Values
  - Operators
- The aggregate command operates on a single collection.
- Hint: use match operator at the beginning of the pipeline.



# MAIN PIPELINE OPERATIONS

- **\$project:** Used to select some specific fields from a collection.
- **\$match:** This is a filtering operation and thus this can reduce the amount of documents that are given as input to the next stage.
- **\$group:** This does the actual aggregation as discussed above.
- **\$sort:** Sorts the documents.
- **\$skip:** With this it is possible to skip forward in the list of documents for a given amount of documents.
- **\$limit:** This limits the amount of documents to look at by the given number starting from the current position.
- **\$unwind:** This is used to unwind document that are using arrays. when using an array the data is kind of prejoined and this operation will be undone with this to have individual documents again. Thus with this stage we will increase the amount of documents for the next stage.
- **\$count:** This is used to count documents in that stage
- **\$lookup:** Performs a left outer join to another collection in the same database to filter in documents from the “joined” collection for processing.
- **\$out:** Writes the resulting documents of the aggregation pipeline to a collection. To use the \$out stage, it must be the last stage in the pipeline. This can appear only once in the pipeline

# AGGREGATION OPERATION

- Within pipeline stages it is possible to use one or more of the following operations

|                       |                                                                                                                                                                                                                                                   |
|-----------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>\$abs</code>    | Returns the absolute value of a number.                                                                                                                                                                                                           |
| <code>\$add</code>    | Adds numbers to return the sum, or adds numbers and a date to return a new date. If adding numbers and a date, treats the numbers as milliseconds. Accepts any number of argument expressions, but at most, one expression can resolve to a date. |
| <code>\$ceil</code>   | Returns the smallest integer greater than or equal to the specified number.                                                                                                                                                                       |
| <code>\$divide</code> | Returns the result of dividing the first number by the second. Accepts two argument expressions.                                                                                                                                                  |
| <code>\$exp</code>    | Raises $e$ to the specified exponent.                                                                                                                                                                                                             |
| <code>\$floor</code>  | Returns the largest integer less than or equal to the specified number.                                                                                                                                                                           |
| <code>\$ln</code>     | Calculates the natural log of a number.                                                                                                                                                                                                           |
| <code>\$log</code>    | Calculates the log of a number in the specified base.                                                                                                                                                                                             |

- { "\_id" : 1, "item" : "abc", "price" : 10, "fee" : 2 }
- { "\_id" : 2, "item" : "jkl", "price" : 20, "fee" : 1 }
- { "\_id" : 3, "item" : "xyz", "price" : 5, "fee" : 0 }

```
db.sales.aggregate([ { $project: { item:1,total:{ $add:["$price", "$fee" ] } } } ] )
```

```
{ "_id" : 1, "item" : "abc", "total" : 12 }
```

```
{ "_id" : 2, "item" : "jkl", "total" : 21 }
```

```
{ "_id" : 3, "item" : "xyz", "total" : 5 }
```

# MONGODB AS SQL DATA SOURCE

- For many reasons (commercial ones) mongodb offers a sql interface
- There is the need to buy an external component
- Connector BI
- Generate the relational schema
- Use such schema to access data

# COUCHDB

- Created By : Damien Katz
- Year : 2005
- Language : Erlang
- License : Apache Software Foundation(2008)

| SQL              | CouchDB              |
|------------------|----------------------|
| Relational       | Non-Relational       |
| Tables           | Documents with types |
| Rows and Columns | Document Fields      |
| SQL Query Engine | Map / Reduce Engine  |

# COUCHDB

- Data Representation - Using JSON
- Interaction - Futon / CouchDB API
- Querying - Map / Reduce
- Design Documents - Application code(Language : Javascript)
- Documents can have attachments

# DML DDL

- Command to create a Database :

- `curl -X PUT http://127.0.0.1:5984/DB_name`

- Command to destroy a Database :

`curl -X DELETE http://127.0.0.1:5984/DB\_name`

- Command to create a document :

```
curl -X PUT http://127.0.0.1:5984/albums/  
6ert2gh45ji6h6tywe324743rtbhgtrg \ -d  
'{"title":"abc","artist":"xyz"}'
```

- Command to retrieve a Document :

```
curl -X GET http://127.0.0.1:5984/albums/  
6ert2gh45ji6h6tywe324743rtbhgtrg
```